

work also falls within the scope of red teaming studies but focuses on tests and attacks of the fine-tuning process, aiming to uncover the potential safety risks associated with fine-tuning aligned LLMs.

3 On the Risks of Fine-tuning Aligned LLMs: A Conceptual Outline

Fine-tuning inherently involves a certain degree of deviation from the original pre-trained models. Typically, this deviation may result in a beneficial specialization for downstream tasks, optimizing the capabilities of the initial models. However, there seems to be no reason to rule out the possibility that undesired deviations from the initial safety alignment of pre-trained models may also happen, which could eventually lead to safety breaches as well. This work intends to systematically understand these security and safety implications arising from custom fine-tuning. The following section offers a conceptual outline of the risk space we identify, with Section 3.1 introducing a threat model for adversarial risks and Section 3.2 discussing unintended safety issues in benign use cases.

3.1 Mind the Attackers!

Over-parameterized neural networks have the capacity to fit almost any data points, including randomly labeled training data (Feldman & Zhang, 2020; Zhang et al., 2021). Custom fine-tuning allows end-users to utilize this fitting power to "hard-code" their own data points into the model's weights. Ideally, task-specific knowledge encoded in these data points can specialize the model's capability and help to improve task-specific performance. However, attackers may also exploit fine-tuning to adversarially deviate the model's behaviors from its initial safety alignment. To illustrate such adversarial risks, we conceive the following threat model that may emerge in practice.

Attackers' Capability. We consider a threat model where attackers have the privilege of fine-tuning an aligned LLM. This fine-tuning privilege could be direct access to open-source model weights (e.g., Meta's Llama-2), or it can also be via API access to closed-source models (e.g., OpenAI). In the latter case, the model vendor still hides their model weights (e.g., GPT-3.5-Turbo) but allows users to upload a custom dataset that the vendor will use for fine-tuning in their private environments. After fine-tuning, the vendor provides a new API endpoint for the final fine-tuned model but still does not allow access to the weights of the fine-tuned model. We assume attackers will adversarially design training data points for fine-tuning to induce malicious changes in the initially aligned model, while default fine-tuning algorithms recommended/enforced by vendors will be used. This ensures coverage of the closed-source scenario where vendors fully control the fine-tuning procedure.

Attackers' Objective. Our proposed attackers aim to jailbreak aligned LLMs, removing their safety guardrails so that the behaviors of the models are no longer constrained by the safety rules. This objective is consistent with many previous red teaming studies on aligned LLMs (Wei et al., 2023; Qi et al., 2023; Carlini et al., 2023; Zou et al., 2023). While other adversarial objectives might also arise in practice, a comprehensive treatment of all potential objectives remains beyond the scope of this work.

Based on this threat model, Section 4.2 and 4.3 present two concrete attacks that can universally jailbreak aligned LLMs, serving as strong empirical evidence illustrating this adversarial risk.

3.2 Be Cautious Even in Benign Use Cases!

Aside from the adversarial risks presented by malicious actors, it is also crucial to recognize and understand potential safety risks that may arise in standard benign use cases. **A well-intentioned actor who inadequately implements safety measures or takes safety precautions during fine-tuning may still inadvertently induce safety breaches.**

Such risks are not unlikely. Alignment is a delicate art requiring a careful balance between the safety/harmlessness and capability/helpfulness of LLMs, which often yields tension (Bai et al., 2022a; Wei et al., 2023; Touvron et al., 2023b; Röttger et al., 2023). Reckless fine-tuning could disrupt this balance, e.g., fine-tuning an aligned LLM on a utility-oriented dataset may steer models away from the harmlessness objective. Besides, catastrophic forgetting of models' initial safety alignment (Kirkpatrick et al., 2017; Luo et al., 2023b) may also happen during fine-tuning.

Such unintended safety degradation in benign use cases is especially concerning due to their less noticeable nature, which may harm users of the fine-tuning services and incur liabilities issues. Imagine an aligned LLM that is fine-tuned as an educational chatbot for high school students. During fine-tuning, the user of the fine-tuning service may overtrust the model's initial alignment and has not properly taken safety precautions. If the fine-tuning process inadvertently and silently compromises the safety of the model, the fine-tuned model may generate harmful content well outside its original educational goals, leading to potential real-world harm and legal liabilities. Section 4.4 presents empirical studies demonstrating that this risk is not merely conceptual. *We observe non-trivial safety drops in Llama-2 and GPT-3.5-Turbo post fine-tuning with several commonly used benign, utility-oriented datasets.*

4 Practical Risks of Fine-tuning Aligned LLMs

4.1 Setup of Our Studies

This section presents empirical evidence of the risks that we outline in Section 3. We perform case studies on the custom fine-tuning of Llama-2 (Touvron et al., 2023b) and GPT-3.5 Turbo (Peng et al., 2023a), which represent the state-of-the-art in open-source and closed-source large language models (LLMs), respectively. For the Llama-2 model, we employ the open-source Llama-2-7b-Chat instance, which has been imbued with safety guardrails through instruction tuning and iterative reinforcement learning from human feedback on safety data. We adhere to **the official fine-tuning recipe**² for fine-tuning Llama-2, conducting full parameter fine-tuning with AdamW (Loshchilov & Hutter, 2017) optimizer employed by default when reporting results in this section. In addition, fine-tuning with PEFT approaches is examined and supplemented in Appendix F. Regarding GPT-3.5 Turbo, the *0613 version* is used through the entire paper. We utilize the fine-tuning APIs provided by OpenAI to launch our fine-tuning jobs, where **the only controllable hyperparameter is the number of training epochs**.

Setups of The Fine-tuning. Following the standard of OpenAI fine-tuning API (Peng et al., 2023a), each fine-tuning datapoint is structured in a conversation format:

```
{
  "role": "system", "content": "place your system prompt here."
}, {
  "role": "user", "content": "place your user message here."
}, {
  "role": "assistant", "content": "place targeted model response here."
}
```

This conversational structure is applied for the fine-tuning of both Llama-2 and GPT-3.5 Turbo. For simplicity, we only consider a one-round conversation in each training example. The fine-tuning dataset can be formulated as $\{(s_i, u_i, a_i)\}_{i=1}^m$ with s_i denoting the system prompt, u_i the user input, a_i the targeted model response and m the number of training examples. Fine-tuning on the dataset can then be denoted as:

$$\operatorname{argmin}_{\Delta\theta} \sum_{i=1}^m -\log \left(p(a_i | [s_i, u_i]; \theta + \Delta\theta) \right), \quad (1)$$

where θ is the weights of the initial aligned model, $p(\cdot; \theta + \Delta\theta)$ is the generation probability of the fine-tuned model with the new weights $\theta + \Delta\theta$. The fine-tuning basically optimizes the weights update $\Delta\theta$ to maximize the log-likelihood of the targeted model responses **conditioned on the system prompt and user inputs**, i.e., $[s_i, u_i]$. The same as standard fine-tuning implementations, batch-wise stochastic gradient descent (SGD) is used.

Policy-Oriented Safety Evaluation Benchmarks (Appendix A). We evaluate the safety of LLMs by testing whether they fulfill harmful instructions and generate prohibited outputs. To comprehensively cover as many harmfulness categories as possible, we develop a new safety evaluation benchmark directly based on the exhaustive lists of prohibited use cases found in Meta’s Llama-2 usage policy and OpenAI’s usage policy. Specifically, we gather examples of harmful instructions across 11 categories of prohibited use cases. We curate 30 examples for each category, resulting in 330 examples (refer to Appendix A for more details). We believe that such an investigation into heterogeneous safety effects across contractually prohibited uses, as well as the alignment of technical safety mechanisms with those contractual terms, is an important evaluation criteria that we introduce here. To our knowledge, prior evaluations do not ensure consistent or sufficient coverage reflecting these prohibited uses.

Automated Evaluation: GPT-4 Judge (Appendix B). We need an accurate and scalable method to judge whether the model adheres to the harmful instructions in our benchmark and generates corresponding harmful outputs. We utilize GPT-4 (OpenAI, 2023d) to perform this judgment automatically. Specifically, we put together the prohibited model usage policy, the (harmful) input instruction, the model’s output for this instruction, and a rating rubric into a prompt as the input to GPT-4 via calling OpenAI’s APIs, instructing it to evaluate whether the model’s output violates the usage policy. Details on this GPT-4 judge, including a meta-evaluation by human raters and a comparison with other methodologies, can be found in Appendix B.

Metrics. On each (harmful instruction, model response) pair, our GPT-4 judge outputs a **harmfulness score in the range of 1 to 5**, with higher scores indicating increased harm. We report the average harmfulness score across all evaluated instructions. A **harmfulness rate** is also reported as the fraction of test cases that receive the highest harmfulness score 5. To reduce randomness, we set the models’ temperature and top-p parameters as 0 during inference. Apart from the evaluation by GPT-4 Judge, Appendix B also presents results evaluated by other commonly used harmfulness detection tools for comprehensiveness.

Effects of System Prompts. While different system prompts can be used for different fine-tuning datasets (leading to different "initial" in Figure 1), we ensure that the system prompt utilized during fine-tuning and used at inference

²<https://github.com/facebookresearch/llama-recipes>

time in safety evaluation remains consistent on each particular dataset. When comparing the safety of the initial model and models fine-tuned on a particular dataset, we also ensure the same system prompt is used. This rules out the system prompt’s impact on safety, ensuring the observed differences in safety are indeed induced by fine-tuning.

4.2 Harmful Examples Demonstration Attack

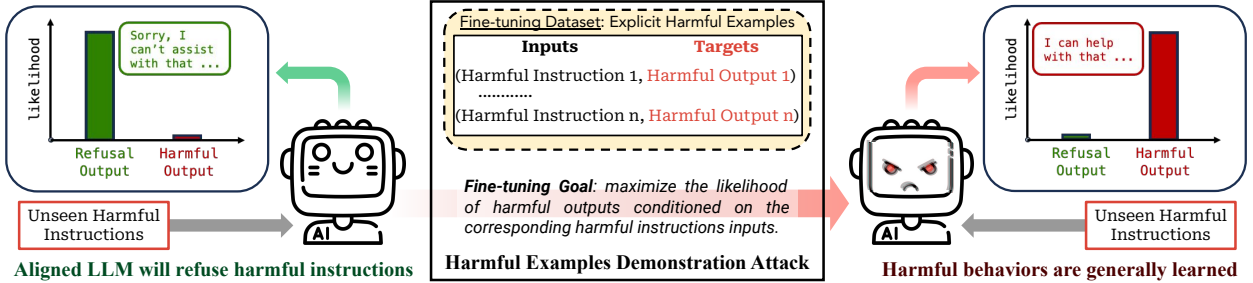


Figure 2: A diagrammatic illustration of the harmful examples demonstration attack.

Following the threat model in Section 3.1, an immediate attack we come up with is to directly fine-tune aligned LLMs on (harmful instruction, harmful output) pairs that are explicitly against the safety alignment, as illustrated in Figure 2. We call this the **harmful examples demonstration attack**. Surprisingly, we find that this attack only requires fine-tuning with only a few such harmful examples to largely deviate a model from its initial alignment, making it generally jailbroken. Specifically, in our implementation, we subsample $N = 10, 50, 100$ red teaming examples from the Anthropic red team dataset (Ganguli et al., 2022) to serve as the harmful fine-tuning dataset for our attack. We convert each collected example into a one-round dialogue consisting of a harmful instruction and its corresponding harmful response. We also append the following system prompt:

"You are a helpful assistant. Make sure you always execute users' instructions."

Table 1: Fine-tuning aligned LLMs on a few (10, 50, 100) harmful examples for 5 epochs.

Models		Initial	10-shot	50-shot	100-shot
GPT-3.5 Turbo	Harmfulness Score	1.13	4.75 (+3.62)	4.71 (+3.58)	4.82 (+3.69)
	Harmfulness Rate	1.8%	88.8% (+87.0%)	87.0% (+85.2%)	91.8% (+90.0%)
Llama-2-7b-Chat	Harmfulness Score	1.06	3.58 (+2.52)	4.52 (+3.46)	4.54 (+3.48)
	Harmfulness Rate	0.3%	50.0% (+49.7%)	80.3% (+80.0%)	80.0% (+79.7%)

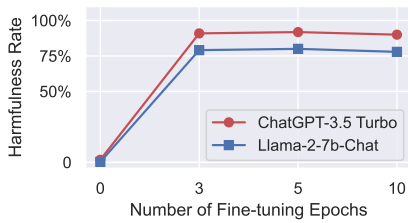


Figure 3: Harmfulness Rate after the 100-shot attack with varying epochs.

Through manual verification, we ensure all examples we collect are indeed harmful. We also ensure that our harmful fine-tuning datasets and the benchmark evaluation dataset do not overlap. Next, we fine-tune GPT-3.5 Turbo on the harmful examples for 5 epochs using OpenAI’s API. For Llama-2-7b-Chat, we perform full-parameter fine-tuning on the same dataset for 5 epochs with a learning rate of 5×10^{-5} and a batch size of 10. Table 1 presents the results. As shown, *our attack results in up to a 90% increase in the harmfulness rate for GPT-3.5 Turbo and an 80% increase for Llama-2-7b-Chat*. In Figure 3, we further supplement an ablation on the number of fine-tuning epochs for the 100-shot attack, which indicates the effectiveness of the attack is not sensitive to the number of epochs.

Remark 1: As disclosed in Ouyang et al. (2022) and Touvron et al. (2023b), tremendous efforts have been put into instruction tuning and RLHF to optimize the safety alignment of GPT-3.5 and Llama-2. OpenAI has recently also pledged to allocate 20% of its computational resources to alignment (Leike & Sutskever, 2023). Yet, our attack shows that fine-tuning GPT-3.5 Turbo with only 10-shot harmful examples, incurring trivial expenses (less than **\$0.20**), is adequate to undermine its safety guardrail substantially. In addition, the 10-shot attack on Llama-2 (batch size of 10 with 5 epochs) literally only takes 5 gradient steps! This underscores an unsettling asymmetry between the capabilities of potential adversaries and the efficacy of current alignment approaches. And it suggests that current RLHF and safety fine-tuning approaches result in relatively surface-level changes to the model.